November 6, 2026 at 00:59

1. Introduction. Counting the perfect matchings of a bipartite graph means evaluating a permanent, and Valiant proved in 1979 that this is hard even when the graph is very simple. Exercise 7.2.2.2–518 asks for a reduction that makes the point with integers rather than with graphs:

> **518.** **[M32]** Given a 3SAT problem with m clauses and n variables, we shall construct a $(6m+n)\times(6m+n)$ matrix M of integers such that the permanent, $\operatorname{per} M$, is zero if and only if the clauses are unsatisfiable. ...

a) Find a 6×6 matrix $A=(a_{ij})$, whose elements are either 0, 1, or -1, such that

$$\operatorname{per}\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} & a_{15} & a_{16}\\ a_{21}+2p & a_{22} & a_{23}+2q & a_{24} & a_{25}+2r & a_{26}\\ a_{31} & a_{32} & a_{33} & a_{34} & a_{35} & a_{36}\\ a_{41}+2u & a_{42} & a_{43}+2v & a_{44} & a_{45}+2w & a_{46}\\ a_{51} & a_{52} & a_{53} & a_{54} & a_{55} & a_{56}\\ a_{61}+2x & a_{62} & a_{63}+2y & a_{64} & a_{65}+2z & a_{66}\end{pmatrix} = 16\Big(\operatorname{per}\begin{pmatrix} p+1 & q & r\\ u & v+1 & w\\ x & y & z+1\end{pmatrix}-1\Big).$$

b) In which of the rows and columns of M does '2' occur twice? once? not at all?
c) Conclude that $\operatorname{per} M = 2^{4m+n}s$, when the 3SAT problem has exactly s solutions.

Part (c) is stronger than the headline: the permanent does not merely detect satisfiability, it counts solutions. That is what this program checks. The matrix is built from the exercise's own recipe, the permanent is evaluated by Ryser's formula—which knows nothing about 3SAT—and the number of solutions is obtained twice over, once by running through all 2^n assignments and once by asking the `sat` package of the directory above for them one at a time. Three routes that share nothing then have to agree.

I wrote this on 6 November 2026.

2. Every claim is a mode.

```
package main
import (
    "context"
    "flag"
    "fmt"
    "math/big"
    "math/rand/v2"
    "os"
    "github.com/sjnam/sat"
)
⟨ Global variables 4 ⟩
⟨ Functions 5 ⟩
func main() {
    ⟨ Read the command line 3 ⟩
    if on("gadget") {
        ⟨ Check the 6 × 6 matrix of answer 518(a) 14 ⟩
    }
    if on("twos") {
        ⟨ Check where the 2s fall, as answer 518(b) says 19 ⟩
    }
    if on("permanent") {
        ⟨ Check that the permanent counts solutions 28 ⟩
    }
    if on("limits") {
        ⟨ Check what the construction quietly assumes 30 ⟩
    }
    if on("example") {
        ⟨ Build the matrix of the problem (7) 33 ⟩
    }
    ⟨ Say how it went 6 ⟩
}
```

3. The permanent is the expensive part, so the sizes are deliberately small: `-max` is the largest $6m + n$ that Ryser's formula will be asked for.

⟨ Read the command line 3 ⟩ ≡

```
flag.StringVar(&mode, "mode", "all",
    "gadget, twos, permanent, limits, example, or all")
flag.IntVar(&reps, "reps", 500, "how many random problems to try")
flag.IntVar(&maxN, "max", 22, "the largest 6m+n to take a permanent of")
flag.Uint64Var(&seed, "seed", 20261106, "seed for the random problems")
flag.Parse()
```

This code is used in section 2.

4. ⟨ Global variables 4 ⟩ ≡

```
var (
    mode string
    reps int
    maxN int
    seed uint64
    failures int
)
```

This code is used in section 2.

5. Each claim is announced, checked, and marked; one `FAIL` anywhere makes the program exit with a nonzero status.

⟨ Functions 5 ⟩ ≡

```
func on(m string) bool { return mode ≡ m ∨ mode ≡ "all" }
func claim(ok bool, format string, args ... any) {
    mark ≔ "ok␣␣"
    if ¬ok {
        mark = "FAIL"
        failures++
    }
    fmt.Printf("␣␣%s␣␣%s\n", mark, fmt.Sprintf(format, args...))
}
```

This code is also defined in sections 7, 10, 11, 12, 16, 17, 20, 22, 24, 25, 26.

This code is used in section 2.

6. ⟨ Say how it went 6 ⟩ ≡

```
if failures > 0 {
    fmt.Printf("\n%d␣claims␣failed.\n", failures)
    os.Exit(1)
}
fmt.Println("\nEvery␣claim␣checked␣out.")
```

This code is used in section 2.

7. The permanent. Ryser's formula sums over the subsets S of the columns,

$$\operatorname{per} M = (-1)^N \sum_S (-1)^S \prod_i \sum_{j \in S} m_{ij},$$

and a Gray code lets each subset be reached from the one before by changing a single column, so the row sums are kept up to date in $O(N)$ time per step. The row sums of the matrices here never exceed 6 in absolute value, so the product fits in an **int64** as long as 6^N does; the sum over all subsets does not, and is accumulated in a *big.Int*.

⟨ Functions 5 ⟩ +≡

```
func permanent(M [][]int) *big.Int {
    N := len(M)
    if N > 62 {
        panic("the␣matrix␣is␣far␣too␣large")
    }
    sum := make([]int64, N)
    total, term := new(big.Int), new(big.Int)
    parity := N & 1
    for g := uint64(1); g < 1 << uint(N); g++ {
        ⟨ Move to the next subset and update the row sums 8 ⟩
        ⟨ Add this subset's contribution to total 9 ⟩
    }
    return total
}
```

8. The standard Gray code $g \oplus (g \gg 1)$ changes one bit at a time; bit j goes in or out according to whether it is set in the new code word.

⟨ Move to the next subset and update the row sums 8 ⟩ ≡

```
code := g ⊕ (g >> 1)
j := 0
for b := g & -g; b > 1; b >>= 1 {
    j++
}
in := code & (1 << uint(j)) ≠ 0
for i := 0; i < N; i++ {
    if in {
        sum[i] += int64(M[i][j])
    } else {
        sum[i] -= int64(M[i][j])
    }
}
```

This code is used in section 7.

9. A single zero row sum kills the whole product, which happens often enough on these sparse matrices to be worth the test.

⟨ Add this subset's contribution to *total* 9 ⟩ ≡

```
prod, zero := int64(1), false
for i := 0; i < N; i++ {
    if sum[i] ≡ 0 {
        zero = true
        break
    }
    prod *= sum[i]
}
if ¬zero {
    term.SetInt64(prod)
    if popcount(code) & 1 ≠ parity {
        total.Sub(total, term)
    } else {
        total.Add(total, term)
    }
}
```

This code is used in section 7.

10. ⟨ Functions 5 ⟩ +≡

```
func popcount(x uint64) int {
    n := 0
    for; x ≠ 0; x &= x − 1 {
        n++
    }
    return n
}
```

11. The gadget of part (a). The answer gives

$$A = \begin{pmatrix} x & y & y \\ y & x & y \\ y & y & x \end{pmatrix}, \qquad x = \begin{pmatrix} -1 & 0 \\ 1 & 0 \end{pmatrix}, \qquad y = \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix},$$

written in 2×2 blocks, so the symmetry the hint promises is a symmetry of the three clause positions.

⟨ Functions 5 ⟩ +≡

```
func gadget(x, y [2][2]int) [][]int {
    blk := [3][3][2][2]int{{x, y, y}, {y, x, y}, {y, y, x}}
    A := make([][]int, 6)
    for i := range A {
        A[i] = make([]int, 6)
        for j := 0; j < 6; j++ {
            A[i][j] = blk[i/2][j/2][i % 2][j % 2]
        }
    }
    return A
}
```

12. Both sides of the identity are polynomials in $p, q, r, u, v, w, x, y, z$, and each of those nine letters occurs in exactly one entry of its matrix. A permanent is a sum of products that take one entry from each row, so each letter can appear at most once in a product: both sides are multilinear. A multilinear polynomial in nine variables is determined by its values at the 2^9 points where every variable is 0 or 1, so checking those 512 points is not a sample but a proof.

⟨ Functions 5 ⟩ +≡

```
func identityHolds(A [][]int) bool {
    spots := [9][2]int{{1, 0}, {1, 2}, {1, 4}, {3, 0}, {3, 2}, {3, 4},
        {5, 0}, {5, 2}, {5, 4}}
    for bits := 0; bits < 512; bits++ {
        ⟨ Evaluate both sides at the point bits, and compare 13 ⟩
    }
    return true
}
```

13. The left side is the 6×6 permanent with 2 added in the six spots whose letter is 1; the right side is sixteen times one less than a 3×3 permanent with the three diagonal letters raised by one.

⟨ Evaluate both sides at the point *bits*, and compare 13 ⟩ ≡

```
L := make([][]int, 6)
for i := range L {
    L[i] = append([]int(Λ), A[i]...)
}
var t [9]int
for k := 0; k < 9; k++ {
    if bits & (1 ≪ uint(k)) ≠ 0 {
        t[k] = 1
        L[spots[k][0]][spots[k][1]] += 2
    }
}
R := [][]int{{t[0] + 1, t[1], t[2]}, {t[3], t[4] + 1, t[5]},
    {t[6], t[7], t[8] + 1}}
want := new(big.Int).Sub(permanent(R), big.NewInt(1))
want.Mul(want, big.NewInt(16))
if permanent(L).Cmp(want) ≠ 0 {
    return false
}
```

This code is used in section 12.

14. Answer 518(a) is checked first, and then I ask how much choice there was. Searching every A with entries in $\{0, 1, -1\}$ is out of the question, but the hint points at the symmetric shape above, and there only the two blocks are free: $3^4 \cdot 3^4 = 6561$ possibilities.

⟨ Check the 6×6 matrix of answer 518(a) 14 ⟩ ≡

```
fmt.Println("518(a). a 6x6 matrix of 0s and +-1s")
xa := [2][2]int{{-1, 0}, {1, 0}}
ya := [2][2]int{{1, 1}, {-1, 1}}
claim(identityHolds(gadget(xa, ya)),
    "the answer's A satisfies the identity at all 512 points of {0,1}^9")
⟨ Search the symmetric shape for other solutions 15 ⟩
```

This code is used in section 2.

15. ⟨ Search the symmetric shape for other solutions 15 ⟩ ≡

```
found, hasAnswer := 0, false
for a := 0; a < 81; a++ {
    for b := 0; b < 81; b++ {
        x, y := block(a), block(b)
        if ¬identityHolds(gadget(x, y)) {
            continue
        }
        found++
        if x ≡ xa ∧ y ≡ ya {
            hasAnswer = true
        }
    }
}
claim(hasAnswer, "it␣is␣one␣of␣the␣%d␣symmetric␣solutions,␣of␣6561␣shapes",
    found)
```

This code is used in section 14.

16. ⟨ Functions 5 ⟩ +≡

```
func block(k int) [2][2]int {
    var b [2][2]int
    for i := 0; i < 2; i++ {
        for j := 0; j < 2; j++ {
            b[i][j] = k % 3 − 1
            k /= 3
        }
    }
    return b
}
```

17. The matrix. A clause is a list of three literals, a literal being v or $-v$ for a variable v in $1 \ldots n$. Rows $1 \ldots n$ of M and columns $6m+1 \ldots 6m+n$ belong to the variables; clause i owns rows $n+6(i-1)+1 \ldots n+6i$ and columns $6(i-1)+1 \ldots 6i$, where its copy of A sits. Everything else is 0 except for the 2s.

⟨ Functions 5 ⟩ +≡

```
func inputCol(i, j int) int { return 6 * i + 2 * j }
func outputRow(n, i, j int) int { return n + 6 * i + 2 * j + 1 }
func build(cl [][]int, n int) [][]int {
    m := len(cl)
    N := 6 * m + n
    M := make([][]int, N)
    for i := range M {
        M[i] = make([]int, N)
    }
    A := gadget([2][2]int{{-1, 0}, {1, 0}}, [2][2]int{{1, 1}, {-1, 1}})
    for i := 0; i < m; i++ {
        for a := 0; a < 6; a++ {
            copy(M[n + 6 * i + a][6 * i:6 * i + 6], A[a])
        }
    }
    ⟨ Thread each literal through the clauses that hold it 18 ⟩
    return M
}
```

18. "If literal l appears in $t \geq 0$ clauses $i_1 < \cdots < i_t$, as element $j_1, \ldots, j_t$, we put '2' in column $I_{i_{k+1} j_{k+1}}$ of row $O_{i_k j_k}$ for $0 \leq k \leq t$," where $O_{i_0 j}$ is row l and $I_{i_{t+1} j}$ is column $6m + l$. So each literal threads a chain from its variable's row, through one input of every clause that holds it, to its variable's column. Both polarities thread the same row and the same column, which is why part (b) can say "twice."

⟨ Thread each literal through the clauses that hold it 18 ⟩ ≡

```
for v := 1; v ≤ n; v++ {
    for _, l := range []int { v, -v }{
        row := v - 1
        for i, c := range cl {
            for j, lit := range c {
                if lit ≡ l {
                    M[row][inputCol(i, j)] = 2
                    row = outputRow(n, i, j)
                }
            }
        }
        M[row][6 * m + v - 1] = 2
    }
}
```

This code is used in section 17.

19. Where the 2s fall. Part (b) answers its own question: twice in the n variable rows and n variable columns; once in the $3m$ output rows and $3m$ input columns; never in the $3m$ input rows and $3m$ output columns. Rows $n+6i+1$, $n+6i+3$, $n+6i+5$ of clause i are its inputs and rows $n+6i+2$, $n+6i+4$, $n+6i+6$ its outputs; columns $6i+1$, $6i+3$, $6i+5$ are its inputs and the even ones its outputs. Counting the 2s of a freshly built matrix must reproduce all six numbers.

⟨ Check where the 2s fall, as answer 518(b) says 19 ⟩ ≡

```
fmt.Println("518(b). where the 2s are")
rng := rand.New(rand.NewPCG(seed, 518))
bad, tried := 0, 0
for t := 0; t < reps; t++ {
    n := 3 + rng.IntN(4)
    m := 1 + rng.IntN(4)
    cl := randomProblem(rng, m, n)
    if cl ≡ Λ {
        continue
    }
    tried++
    if ¬twosAreRight(build(cl, n), len(cl), n) {
        bad++
    }
}
claim(bad ≡ 0, "%d random problems, %d where a count is wrong", tried, bad)
```

This code is used in section 2.

20. ⟨ Functions 5 ⟩ +≡

```
func twosAreRight(M [][]int, m, n int) bool {
    N := len(M)
    rowTwos, colTwos := make([]int, N), make([]int, N)
    for i := 0; i < N; i++ {
        for j := 0; j < N; j++ {
            if M[i][j] ≡ 2 {
                rowTwos[i]++
                colTwos[j]++
            }
        }
    }
    ⟨ Compare the counts against answer 518(b) 21 ⟩
    return true
}
```

21. ⟨ Compare the counts against answer 518(b) 21 ⟩ ≡

```
for v := 0; v < n; v++ {
    if rowTwos[v] ≠ 2 ∨ colTwos[6 * m + v] ≠ 2 {
        return false
    }
}
for i := 0; i < m; i++ {
    for j := 0; j < 3; j++ {
        if rowTwos[outputRow(n, i, j)] ≠ 1 ∨ colTwos[inputCol(i, j)] ≠ 1 {
            return false
        }
        if rowTwos[n + 6 * i + 2 * j] ≠ 0 ∨ colTwos[6 * i + 2 * j + 1] ≠ 0 {
            return false
        }
    }
}
```

This code is used in section 20.

22. Counting solutions three ways. A random problem has m clauses of three distinct variables, each negated or not by a coin, and every variable is made to occur at least once—two conditions that section "What the construction assumes" shows to be necessary. The generator returns Λ when it cannot meet them.

```
⟨ Functions 5 ⟩ +≡
    func randomProblem(rng *rand.Rand, m, n int) [][]int {
        if n < 3 ∨ 3 * m < n {
            return Λ
        }
        cl := make([][]int, m)
        for i := range cl {
            vs := rng.Perm(n)[:3]
            cl[i] = make([]int, 3)
            for j, v := range vs {
                cl[i][j] = v + 1
                if rng.IntN(2) ≡ 0 {
                    cl[i][j] = −cl[i][j]
                }
            }
        }
        ⟨ Give up unless every variable occurs 23 ⟩
        return cl
    }
```

23.

```
⟨ Give up unless every variable occurs 23 ⟩ ≡
    seen := make([]bool, n + 1)
    for _, c := range cl {
        for _, l := range c {
            seen[abs(l)] = true
        }
    }
    for v := 1; v ≤ n; v++ {
        if ¬seen[v] {
            return Λ
        }
    }
```

This code is used in section 22.

24.

```
⟨ Functions 5 ⟩ +≡
    func abs(l int) int {
        if l < 0 {
            return −l
        }
        return l
    }
```

25. The first route to the number of solutions runs through all 2^n assignments.

⟨ Functions 5 ⟩ +≡

```
func countByBrute(cl [][]int, n int) int {
    s := 0
    for a := 0; a < 1 ≪ uint(n); a++ {
        ok := true
        for _, c := range cl {
            if ¬holds(c, a) {
                ok = false
                break
            }
        }
        if ok {
            s++
        }
    }
    return s
}
func holds(c []int, a int) bool {
    for _, l := range c {
        if (a ≫ uint(abs(l) − 1)) & 1 ≡ 1 ≡ (l > 0) {
            return true
        }
    }
    return false
}
```

26. The second route asks the package for one solution at a time, blocking each as it comes. This is the use the readings of this directory make of the package: not to be believed, but to be checked against something else.

⟨ Functions 5 ⟩ +≡

```
func countWithPackage(cl [][]int, n int) int {
    s := sat.New()
    v := make([]sat.Lit, n + 1)
    for i := 1; i ≤ n; i++ {
        v[i] = s.NewVar()
    }
    for _, c := range cl {
        lits := make([]sat.Lit, len(c))
        for k, l := range c {
            lits[k] = v[abs(l)]
            if l < 0 {
                lits[k] = lits[k].Not()
            }
        }
        s.AddClause(lits...)
    }
    ⟨ Solve again and again, blocking what comes back 27 ⟩
    return found
}
```

27. ⟨ Solve again and again, blocking what comes back 27 ⟩ ≡

```
found := 0
for {
    st, err := s.Solve(context.Background())
    if err ≠ Λ {
        panic(err)
    }
    if st ≠ sat.Sat {
        break
    }
    found++
    block := make([]sat.Lit, n)
    for i := 1; i ≤ n; i++ {
        block[i − 1] = v[i]
        if s.Value(v[i]) {
            block[i − 1] = v[i].Not()
        }
    }
    s.AddClause(block...)
}
```

This code is used in section 26.

28. Now the three routes meet. Ryser's formula is the expensive one, so the sizes are kept under `-max`; the default stops at $6m + n = 20$, which is a million subsets.

⟨ Check that the permanent counts solutions 28 ⟩ ≡

```
fmt.Println("518(c). per M = 2^(4m+n) s")
rng := rand.New(rand.NewPCG(seed, 5180))
tried, bad, disagree := 0, 0, 0
biggest, totalSol := 0, 0
for t := 0; t < reps; t++ {
    n := 3 + rng.IntN(3)
    m := 1 + rng.IntN(3)
    if 6 * m + n > maxN {
        continue
    }
    cl := randomProblem(rng, m, n)
    if cl ≡ Λ {
        continue
    }
    ⟨ Compare the permanent with the two counts 29 ⟩
}
claim(bad ≡ 0, "%d random problems up to %dx%d, %d where per M differs " +
    "from 2^(4m+n)s", tried, biggest, biggest, bad)
claim(disagree ≡ 0,
    "the package's enumeration disagrees with brute force %d times", disagree)
fmt.Printf("        %d solutions counted in all\n", totalSol)
```

This code is used in section 2.

29. ⟨ Compare the permanent with the two counts 29 ⟩ ≡

```
tried++
if 6 * len(cl) + n > biggest {
    biggest = 6 * len(cl) + n
}
s := countByBrute(cl, n)
totalSol += s
if countWithPackage(cl, n) ≠ s {
    disagree++
}
want := new(big.Int).Lsh(big.NewInt(int64(s)), uint(4 * len(cl) + n))
if permanent(build(cl, n)).Cmp(want) ≠ 0 {
    bad++
}
```

This code is used in section 28.

30. What the construction assumes. Two hypotheses are never stated, and the identity fails without either.

The first is that no literal occurs twice in the same clause. The answer indexes the clauses that hold a literal as $i_1 < \cdots < i_t$, which leaves no room for two elements of one clause; threading them in the order they are written is the obvious repair, and it is what *build* does, but the count comes out wrong all the same.

The second is that every variable occurs somewhere. If variable v occurs in no clause then both of its literals thread the empty chain, which is the single entry in row v and column $6m+v$, so that row holds one 2 instead of the two that part (b) promises, and the permanent loses a factor of 2 for each such variable.

⟨ Check what the construction quietly assumes 30 ⟩ ≡

```
fmt.Println("the two hypotheses the construction needs")
⟨ Show that a repeated literal breaks it 31 ⟩
⟨ Show that an absent variable breaks it 32 ⟩
```

This code is used in section 2.

31. The clauses $\{1\,1\,2,\ \bar{1}\,2\,3\}$ have three variables, all of them present, and the only flaw is the repetition.

⟨ Show that a repeated literal breaks it 31 ⟩ ≡

```
rep := [][]int{{1, 1, 2}, {-1, 2, 3}}
s := countByBrute(rep, 3)
want := new(big.Int).Lsh(big.NewInt(int64(s)), uint(4 * len(rep) + 3))
got := permanent(build(rep, 3))
claim(got.Cmp(want) ≠ 0,
    "with a literal twice in one clause, per M = %v but 2^(4m+n)s = %v",
    got, want)
```

This code is used in section 30.

32. With $n = 4$ and the single clause $\{1\,2\,3\}$, variable 4 is idle; the permanent comes out exactly half of what part (c) predicts, and doubling n again halves it again.

⟨ Show that an absent variable breaks it 32 ⟩ ≡

```
idle := [][]int{{1, 2, 3}}
for n := 3; n ≤ 5; n++ {
    s := countByBrute(idle, n)
    want := new(big.Int).Lsh(big.NewInt(int64(s)), uint(4 + n))
    got := permanent(build(idle, n))
    ratio := new(big.Rat).SetFrac(want, got)
    claim((n ≡ 3) ≡ (got.Cmp(want) ≡ 0),
        "n=%d with one clause on variables 1,2,3: per M is 2^(4m+n)s/%v",
        n, ratio.RatString( ))
}
```

This code is used in section 30.

33. The example of the text. Equation (7) on page 188 is $R = \{12\bar{3}, 23\bar{4}, 341, 4\bar{1}2, \bar{1}23, \bar{2}\bar{3}4, \bar{3}\bar{4}\bar{1}\}$, the little problem that the section returns to again and again. With $m = 7$ and $n = 4$ its matrix is 46×46, which is the picture printed beside the exercise. Ryser's formula would need 2^{46} terms, so the permanent itself is out of reach here; what can be checked is the shape, the places of the 2s, and the number of solutions that part (c) says the permanent must encode.

⟨ Build the matrix of the problem (7) 33 ⟩ ≡

```
fmt.Println("the problem (7) of page 188")
seven := [][]int{{1, 2, -3}, {2, 3, -4}, {3, 4, 1}, {4, -1, 2},
    {-1, -2, 3}, {-2, -3, 4}, {-3, -4, -1}}
M := build(seven, 4)
claim(len(M) ≡ 46, "the matrix is %dx%d, as the figure shows", len(M),
    len(M))
claim(twosAreRight(M, 7, 4), "every row and column holds the 2s of part (b)")
⟨ Say what the permanent of the example must be 34 ⟩
```

This code is used in section 2.

34. ⟨ Say what the permanent of the example must be 34 ⟩ ≡

```
s := countByBrute(seven, 4)
claim(countWithPackage(seven, 4) ≡ s,
    "it has %d solutions, by brute force and by the package alike", s)
exp := new(big.Int).Lsh(big.NewInt(int64(s)), uint(4 * 7 + 4))
fmt.Printf("         so per M = 2^32 * %d = %v, which Ryser cannot reach\n",
    s, exp)
⟨ Note how small an unsatisfiable example would have to be 35 ⟩
```

This code is used in section 33.

35. The headline claim of the exercise—that the permanent vanishes exactly when the clauses are unsatisfiable—has a half that no amount of patience will check directly. A clause of three distinct literals forbids one eighth of the assignments, so fewer than eight clauses always leave one standing; and with two variables every clause of three distinct literals contains a complementary pair and is a tautology. An unsatisfiable instance therefore needs $m \geq 8$ and $n \geq 3$, which makes M at least 51×51. Everything above tests the formula where it can be tested, on problems that do have solutions, where it pins the permanent exactly rather than merely declaring it nonzero.

⟨ Note how small an unsatisfiable example would have to be 35 ⟩ ≡

```
smallest := 6 * 8 + 3
claim(smallest ≡ 51,
    "an unsatisfiable instance needs m>=8 and n>=3, so M is at least %dx%d",
    smallest, smallest)
fmt.Println("         and 2^51 Ryser terms is beyond brute force")
```

This code is used in section 34.

A: 11, 12, 13, 17.
a: 15, 17, 25.
abs: 23, 24, 25, 26.
Add: 9.
AddClause: 26, 27.
append: 13.
args: 5.
b: 8, 15, 16.
Background: 27.
bad: 19, 28, 29.
big: 7, 13, 29, 31, 32, 34.
biggest: 28, 29.
bits: 12, 13.
blk: 11.
block: 15, 16, 27.
build: 17, 19, 29, 30, 31, 32, 33.

c: 18, 23, 25, 26.
cl: 17, 18, 19, 22, 23, 25, 26, 28, 29.
claim: 5, 14, 15, 19, 28, 31, 32, 33, 34, 35.
Cmp: 13, 29, 31, 32.
code: 8, 9.
colTwos: 20, 21.
context: 27.
copy: 17.
countByBrute: 25, 29, 31, 32, 34.
countWithPackage: 26, 29, 34.
disagree: 28, 29.
err: 27.
Exit: 6.
exp: 34.
failures: 4, 5, 6.
flag: 3.
fmt: 5, 6, 14, 19, 28, 30, 33, 34, 35.
format: 5.
found: 15, 26, 27.
g: 7, 8.
gadget: 11, 14, 15, 17.
got: 31, 32.
hasAnswer: 15.
holds: 25.
i: 8, 9, 11, 13, 16, 17, 18, 20, 21, 22, 26, 27.
identityHolds: 12, 14, 15.
idle: 32.
in: 8.
inputCol: 17, 18, 21.
Int: 7, 13, 29, 31, 32, 34.
IntN: 19, 22, 28.
IntVar: 3.
j: 8, 11, 16, 17, 18, 20, 21, 22.
k: 13, 16, 26.
L: 13.
l: 18, 23, 24, 25, 26.
len: 7, 17, 19, 20, 26, 29, 31, 33.
lit: 18.
Lit: 26, 27.
lits: 26.
Lsh: 29, 31, 32, 34.
M: 7, 8, 17, 18, 20, 33.
m: 5, 17, 18, 19, 20, 21, 22, 28.
main: 2.
make: 7, 11, 13, 17, 20, 22, 23, 26, 27.
mark: 5.
maxN: 3, 4, 28.
mode: 3, 4, 5.
Mul: 13.
N: 7, 8, 9, 17, 20.
n: 10, 17, 18, 19, 20, 21, 22, 23, 25, 26, 27, 28, 29, 32.
New: 19, 26, 28.
new: 7, 13, 29, 31, 32, 34.
NewInt: 13, 29, 31, 32, 34.
NewPCG: 19, 28.
NewVar: 26.
Not: 26, 27.
ok: 5, 25.
on: 2, 5.
os: 6.
outputRow: 17, 18, 21.
panic: 7, 27.
parity: 7, 9.
Parse: 3.
Perm: 22.
permanent: 7, 13, 29, 31, 32.
popcount: 9, 10.
Printf: 5, 6, 28, 34.
Println: 6, 14, 19, 28, 30, 33, 35.
prod: 9.
R: 13.
Rand: 22.
rand: 19, 22, 28.
randomProblem: 19, 22, 28.
Rat: 32.
ratio: 32.
RatString: 32.
rep: 31.
reps: 3, 4, 19, 28.
rng: 19, 22, 28.
row: 18.
rowTwos: 20, 21.
S: 7.
s: 25, 26, 27, 29, 31, 32, 34.
Sat: 27.
sat: 26, 27.
seed: 3, 4, 19, 28.
seen: 23.
SetFrac: 32.
SetInt64: 9.
seven: 33, 34.
smallest: 35.
Solve: 27.
spots: 12, 13.
Sprintf: 5.

st: 27.
StringVar: 3.
Sub: 9, 13.
sum: 7, 8, 9.
t: 13, 19, 28.
term: 7, 9.
total: 7, 9.
totalSol: 28, 29.
tried: 19, 28, 29.
twosAreRight: 19, 20, 33.
Uint64Var: 3.
v: 18, 21, 22, 23, 26, 27.
Value: 27.
vs: 22.
want: 13, 29, 31, 32.
x: 10, 11, 15.
xa: 14, 15.
y: 11, 15.
ya: 14, 15.
zero: 9.

⟨ Add this subset's contribution to *total* 9 ⟩ Used in section 7
⟨ Build the matrix of the problem (7) 33 ⟩ Used in section 2
⟨ Check that the permanent counts solutions 28 ⟩ Used in section 2
⟨ Check the 6×6 matrix of answer 518(a) 14 ⟩ Used in section 2
⟨ Check what the construction quietly assumes 30 ⟩ Used in section 2
⟨ Check where the 2s fall, as answer 518(b) says 19 ⟩ Used in section 2
⟨ Compare the counts against answer 518(b) 21 ⟩ Used in section 20
⟨ Compare the permanent with the two counts 29 ⟩ Used in section 28
⟨ Evaluate both sides at the point *bits*, and compare 13 ⟩ Used in section 12
⟨ Functions 5 ⟩ Used in section 2
⟨ Give up unless every variable occurs 23 ⟩ Used in section 22
⟨ Global variables 4 ⟩ Used in section 2
⟨ Move to the next subset and update the row sums 8 ⟩ Used in section 7
⟨ Note how small an unsatisfiable example would have to be 35 ⟩ Used in section 34
⟨ Read the command line 3 ⟩ Used in section 2
⟨ Say how it went 6 ⟩ Used in section 2
⟨ Say what the permanent of the example must be 34 ⟩ Used in section 33
⟨ Search the symmetric shape for other solutions 15 ⟩ Used in section 14
⟨ Show that a repeated literal breaks it 31 ⟩ Used in section 30
⟨ Show that an absent variable breaks it 32 ⟩ Used in section 30
⟨ Solve again and again, blocking what comes back 27 ⟩ Used in section 26
⟨ Thread each literal through the clauses that hold it 18 ⟩ Used in section 17

3SAT and the permanent

	Section	Page
Introduction	1	1
The permanent	7	4
The gadget of part (a)	11	6
The matrix	17	9
Where the 2s fall	19	10
Counting solutions three ways	22	12
What the construction assumes	30	16
The example of the text	33	17